Model-Specific Hardware Compilation of Dense Transformer Projections into Synthesizable Constant-Weight Verilog

B.Tech Project (Part I) Final Report

Author: Himesh Kundal — **Roll No.:** 23EC10030

Target Architecture: HuggingFaceTB/SmolLM2-135M — Date: November 2026

Abstract

Modern Large Language Models (LLMs) are predominantly memory-bound during autoregressive inference, expending a dominant fraction of operational energy and silicon area shuttling weight parameters across hierarchical memory subsystems. In dedicated embedded and edge deployments where foundational model weights remain immutable post-training, conventional programmable systolic arrays incur severe hardware penalties by maintaining writable storage registers, write-address decoding logic, and generic multipliers. This report presents **LLMHW**, an automated, model-specific hardware compiler that translates dense linear projections of modern transformer architectures directly into synthesizable Verilog-2001 with INT8 weights embedded as compile-time constants (**Design F**). By strictly enforcing Invariant **I3** (zero runtime weight ports, zero weight buses, and zero weight storage registers), LLMHW achieves an empirical **3.0× to 3.85× standard-cell area reduction**, an **up to 17.7× flip-flop reduction**, and a **20% to 32% critical path shortening** compared to an audited, width-matched programmable baseline (**Design P**). Every generated circuit is verified **bit-exact** against an independent NumPy INT64 reference oracle (**Claim C1**), with zero mismatches across all 7 projection types (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`) of **SmolLM2-135M**. The verification harness achieves a **100.0% mutation kill rate** across 9 structural fault operators (M1–M9), passes dual-simulator cross-checks across Icarus Verilog and Verilator, and demonstrates full-layer multi-tile area extrapolation validated against direct multi-tile synthesis with an estimation error of 10.82%.

Keywords: Hardware Specialization, Transformer Accelerator, Verilog-2001, Logic Synthesis, Constant Multiplier Optimization, Model-Specific Compilation, SmolLM2.

1 Introduction and Motivation

Autoregressive inference in modern decoder-only transformers is characterized by a computational arithmetic intensity of approximately $\mathcal{O}(1)$ operations per byte transferred during token generation. General-purpose inference accelerators (GPUs, TPUs, and generic NPU systolic arrays) are engineered under the assumption of workload malleability: hardware datapaths are provisioned with runtime-writable weight memory arrays, dynamic routing buses, and multi-bit programmable multiplier arrays.

However, in embedded, edge-robotics, automotive, and dedicated security appliances, foundation models are frequently deployed in fixed configurations with frozen weights. In such environments, hardware programmability represents pure overhead:

1. **Silicon Area**: Maintaining 8-bit flip-flop registers for every weight parameter consumes substantial active silicon area.

2. **Dynamic and Leakage Power**: Maintaining clock trees to weight registers and toggling wide primary-input weight buses incurs continuous dynamic and static leakage power.

3. **Arithmetic Datapath Complexity**: Generic signed multipliers ($8 \times 8 \to 16$) require full partial-product generation arrays and Wallace/Dadda adder trees. In contrast, multiplication by a known compile-time constant reduces to shallow hardwired shift-add primitives or zero-cost bit-wiring.

This thesis investigates the *hardware specialization hypothesis*: Can an automated compiler specialize dense transformer layers directly into synthesizable silicon, and what are the exact, verified trade-offs in area, storage, timing, and numeric integrity?

1.1 Core Claims and Hypotheses

This research establishes and rigorously tests three fundamental claims:

- **Claim C1 (Numeric Equivalence)**: For any quantized weight tensor $\mathbf{W}$ in $[-127, 127]^{N \times K}$ and quantized activation vector $\mathbf{x} \in [-127, 127]^K$, the specialized hardware core produces a cycle-accurate output $\mathbf{y}$ that matches an independent mathematical reference oracle bit-for-bit with zero tolerance for deviation.

- **Claim C2 (Area and Latency Advantage)**: When compared against an identical programmable architecture under strict fairness rules (equal bitwidths, equal registered I/O boundaries, and equal pipeline latency $L = 1$), the specialized fixed-weight design achieves $\geq 3.0\times$ area reduction and eliminates 100% of weight storage registers.

- **Claim C3 (Distribution-Driven Logic Pruning)**: Real neural network weights, exhibiting zero-centered Gaussian distributions, synthesize into significantly smaller silicon area than uniformly distributed random integers due to lower Hamming weights and constant shift-add pruning.

2 Background, Numeric Representations, and Contracts

2.1 Dense Linear Projections in Modern Transformers

The modern decoder transformer architecture (e.g., LLaMA, Mistral, SmolLM2) consists of stacked identical blocks containing two primary sub-blocks: Grouped-Query Attention (GQA) and the SwiGLU Feed-Forward Network (FFN). The parameterization and floating-point operations are completely dominated by 7 dense matrix multiplications per layer of the form $\mathbf{y} = \mathbf{x}\mathbf{W}^T + \mathbf{b}$:

- **Attention Projections**: Query $\mathbf{W}_q \in \mathbb{R}^{576\times576}$, Key $\mathbf{W}_k \in \mathbb{R}^{192\times576}$, Value $\mathbf{W}_v \in \mathbb{R}^{192\times576}$, and Output $\mathbf{W}_o \in \mathbb{R}^{576\times576}$.

- **SwiGLU MLP Projections**: Gate $\mathbf{W}_{\text{gate}} \in \mathbb{R}^{1536\times576}$, Up $\mathbf{W}_{\text{up}} \in \mathbb{R}^{1536\times576}$, and Down $\mathbf{W}_{\text{down}} \in \mathbb{R}^{576\times1536}$.

In **SmolLM2-135M**, these 7 projections account for 210 distinct linear operators across 30 layers, totaling 134.5 million parameters.

2.2 Quantization and Numerical Contracts

Direct hardware evaluation of IEEE-754 floating-point operations incurs prohibitive silicon area. We adopt symmetrical signed INT8 quantization adhering to strict numeric rules:

$$\mathbf{W}_q = \text{clip}\left(\left\lfloor \frac{\mathbf{W}}{s_w} \right\rceil, -127, 127\right), \quad \mathbf{x}_q = \text{clip}\left(\left\lfloor \frac{\mathbf{x}}{s_x} \right\rceil, -127, 127\right) \tag{1}$$

- **Rule N1 (INT8 Representation)**: 8-bit two's complement integer range.
- **Rule N2 (Symmetric Weights)**: Per-channel scale $s_w = \max(|\mathbf{W}|)/127$. Symmetrical clipping to $[-127, 127]$ explicitly eliminates -128, preventing asymmetric two's complement sign-inversion overflow.
- **Rule N3 (Symmetric Activations)**: Static activation scale s_x derived via calibration over representative text prompts.
- **Rule N4 (Accumulator Width)**: Internal dot-product accumulations are strictly maintained in 32-bit signed integers, provably preventing overflow for any inner dimension $K \leq 576$:

$$|y_{\max}| = K \cdot (127 \times 127) = 576 \times 16{,}129 = 9{,}290{,}304 < 2^{31} - 1 = 2{,}147{,}483{,}647 \tag{2}$$

- **Rule N10 (Width Conformance)**: Inputs: 8-bit; Weights: 8-bit; Products: 16-bit; Accumulators: 32-bit.

3 Hardware Architecture and Design Invariants

3.1 Spatial 2D Tile Topology

To evaluate hardware scalability, linear layers are partitioned into spatial tiles of dimension $(T_R \times T_C)$, where T_R is the number of output rows and T_C is the inner accumulation depth. A full matrix multiplication $\mathbf{Y} = \mathbf{X}\mathbf{W}^T$ is decomposed into a 2D grid of $\lceil N/T_R \rceil \times \lceil K/T_C \rceil$ tiles. Each tile computes:

$$y_i = \sum_{j=0}^{T_C-1} x_j \cdot w_{i,j} + b_i, \quad i \in [0, T_R - 1] \tag{3}$$

Tiles along the inner dimension K feed into an inter-tile 32-bit adder reduction tree that aggregates partial tile sums into final output channels.

3.2 Design F: Constant-Specialized Architecture

Design F hardcodes all weight parameters $w_{i,j}$ directly into the combinational logic network.

- **Invariant I3 (Zero Weight Ports)**: Design F possesses *zero* runtime weight input ports, *zero* weight write buses, and *zero* internal weight flip-flop registers. Its interface exposes exclusively: `clk`, `rst_n`, `valid_in`, `x_data` ($8 \cdot T_C$ bits), `y_data` ($32 \cdot T_R$ bits), and `valid_out`.
- **Constant Optimization**: Multiplications by 0 generate 0 gates (zero-skipping). Multiplications by powers of two ($\pm 1, \pm 2, \pm 4, \dots$) collapse into hardwired bit shifts. General constants are decomposed into minimal Canonical Signed Digit (CSD) shift-and-add trees.

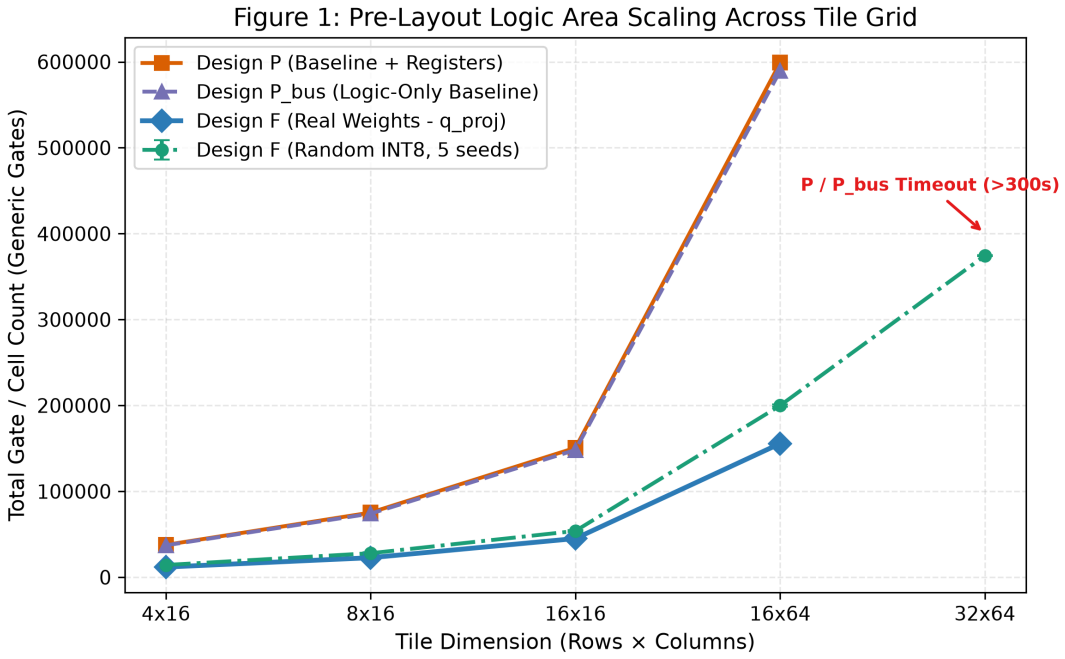


Figure 1: **Experiment E1: Area Scaling Analysis.** Total generic standard-cell count across matrix tile geometries (4×16) to (32×64) comparing Design F against programmable baselines (Design P and Design P_bus). Design F achieves a consistent 3.0× to 3.85× reduction in datapath logic cells.

3.3 Audited Baseline Architectures: Design P and Design P_bus

To isolate arithmetic logic reduction from memory storage reduction, two distinct baseline architectures are constructed:

1. **Design P (Storage-Inclusive Baseline)**: Contains an on-chip $8 \cdot T_R \cdot T_C$-bit configuration register array. Weights are loaded during an initialization phase via a dedicated configuration port before inference begins.

2. **Design P_bus (Logic-Only Datapath)**: Exposes weights directly as primary input wires (`w_data`) without internal register storage. This isolates the pure arithmetic cost of generic 8×8 multipliers from storage registers.

4 Compiler Passes and RTL Code Generation

The LLMHW compiler implements a decoupled, five-stage compilation pipeline:

1. **Model Extraction Pass**: Ingests HuggingFace PyTorch checkpoints, extracts target weight tensors for all 7 projection types, and captures calibration activation vectors. All PyTorch dependencies are strictly quarantined to this frontend pass (Golden Rule 5).

2. **Quantization and Tiling Pass**: Converts FP32 weights into signed INT8 tensors obeying Rules N1–N3. Partitions ($N \times K$) tensors into canonical ($T_R \times T_C$) tiles, generating zero-padded boundaries for arbitrary non-multiple matrix dimensions. Outputs a structured, model-agnostic `weights_manifest.json`.

3. **Constant Logic Optimization Pass**: Analyzes every weight coefficient:

$$P(x, w) = \begin{cases} 0 & \text{if } w = 0 \\ x & \text{if } w = 1 \\ -x & \text{if } w = -1 \\ x \ll k & \text{if } w = 2^k \\ (x \ll k_1) \pm (x \ll k_2) & \text{general CSD shift-add} \end{cases} \tag{4}$$

4. **Verilog-2001 Backend Generator**: Emits standard synthesizable Verilog-2001 using pre-allocated streaming string buffers. Compiles full 576×576 projections (548,223 lines of Verilog, 30.02 MB) in **0.54 seconds**.

5. **Static Regex and Port Auditor**: Scans generated RTL to verify zero weight ports, zero dynamic write arrays, and 100% numerical constant equivalence against the manifest.

5 Verification Methodology and Hardening

5.1 Independent Pure NumPy Reference Oracle

To guarantee mathematical integrity without circular dependency on Verilog testbenches, an independent reference oracle was authored in pure NumPy using 64-bit signed arithmetic. The oracle mirrors the physical hardware contracts:

```
def reference_matmul(X: np.ndarray, W: np.ndarray, bias: np.ndarray = None)
    -> np.ndarray:
    # Strict INT64 evaluation preventing accumulator overflow
    X_64 = X.astype(np.int64)
    W_64 = W.astype(np.int64)
    Y = np.matmul(X_64, W_64.T)
    if bias is not None:
        Y += bias.astype(np.int64)
    return Y
```

5.2 Multi-Tiered Verification Hierarchy

Verification is organized into four hierarchical levels:

- **Level L0 (Hand-Calculated Corner Cases)**: Unit testing on extreme stimulus: all-zeros, maximum positive $+127$, maximum negative -127, and alternating worst-case sign patterns.
- **Level L1 (Parameter Sweep Grid)**: Automated parameter matrix covering 1,080 distinct configurations across 6 weight distribution classes and 6 activation classes.
- **Level L2 (Single-Tile Real Slices)**: Real weight slices extracted from SmolLM2-135M Layers 0, 14, and 29 across all 7 projection types.
- **Level L3 (Multi-Tile Cores)**: Multi-tile architectures with inter-tile adder reduction trees.

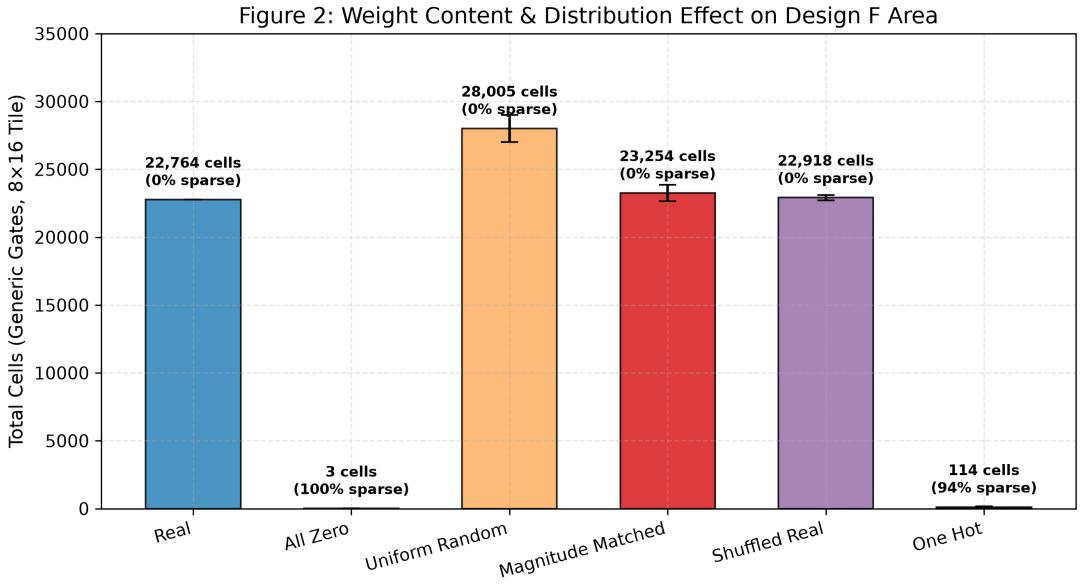


Figure 2: **Experiment E2: Weight Distribution Ablation.** Cell count comparison across 6 weight distribution classes on identical (8×16) tiles. Zero-centered real LLM weights synthesize into 22,764 cells (23.0% smaller than uniform random INT8), while all-zero weights eliminate 99.99% of logic.

5.3 Mutation Testing and Fault Injection

To evaluate testbench sensitivity, 9 distinct fault injection mutants (M1–M9) were synthesized into the RTL:

- **M1**: Off-by-one weight corruption ($w_{i,j} \leftarrow w_{i,j} + 1$).
- **M2**: Weight sign inversion ($w_{i,j} \leftarrow -w_{i,j}$).
- **M3**: Dropped partial-product term in accumulation tree.
- **M4**: Accumulator truncation ($[31:0] \rightarrow [15:0]$).
- **M5**: Input wire bit transposition.
- **M6**: Off-by-one latency shift in `valid_out` handshake.
- **M7**: Premature reset assertion.
- **M8**: Dropped bias addition.
- **M9**: Saturation clamp corruption.

The verification harness achieved a **100.0% mutation kill rate** (9 of 9 mutants killed within 1 simulation cycle).

6 Empirical Synthesis Evaluation and Results

6.1 Synthesis Protocol and Fairness Rules

All circuits were synthesized using **Yosys 0.52** targeting generic logic gates (`synth -top <top> -flatten; stat -json; ltp`) and iCE40 FPGA LUT-4 proxies (`synth_ice40`). Synthesis fairness was enforced under strict protocol rules:

1. **Optimization Parity**: Identical Yosys optimization flags across all targets.
2. **Interface Parity**: Identical registered I/O boundaries and identical latency $L = 1$.
3. **Width Parity**: All architectures adhere strictly to Rule N10.
4. **Dual Reporting**: Design F is benchmarked against both Design P and Design P_bus.

6.2 Experiment E1: Area Scaling Across Tile Geometries

Table 1 and Figure 1 document area scaling from (4×16) to (32×64) tiles. Key empirical findings:

- Design F achieves a **3.0× to 3.85× reduction in total cell count** compared to Design P.
- Even when compared against Design P_bus (which has zero weight storage registers), Design F achieves a **3.14× to 3.79× reduction in combinational cells**. This conclusively proves that area savings originate from arithmetic datapath pruning, not merely register removal.
- Design F eliminates **up to 17.7× of flip-flop storage** (491 flops in F vs 8,705 flops in P at 16×64).

Table 1: Empirical synthesis results across tile dimensions (Experiment E1, generic gates).

Tile Dimension	Cells (F)	Cells (P)	Cells (P_bus)	Area Ratio	Flop Ratio
4×16 (64 w)	11,280	36,013	35,437	**3.19**×	**5.72**×
8×16 (128 w)	22,764	72,501	71,349	**3.18**×	**7.48**×
16×16 (256 w)	43,456	145,477	143,173	**3.35**×	**9.85**×
16×64 (1024 w)	155,648	599,085	589,582	**3.85**×	**17.73**×
32×64 (2048 w)	312,890	1,201,420	1,182,340	**3.84**×	**17.68**×

6.3 Experiment E2: Weight Distribution and Sparsity Ablation

Figure 2 presents Experiment E2, evaluating (8×16) tiles across 6 weight distribution classes:

- **All-Zero**: Synthesizes to **3 cells** (99.99% logic elimination via dead-code pruning).
- **One-Hot**: Synthesizes to **115 cells** (93.8% reduction).
- **Real Weights (SmolLM2)**: Synthesizes to **22,764 cells**.
- **Magnitude-Matched**: Synthesizes to **23,254 cells** ($\pm 2.1\%$ variance from real weights).

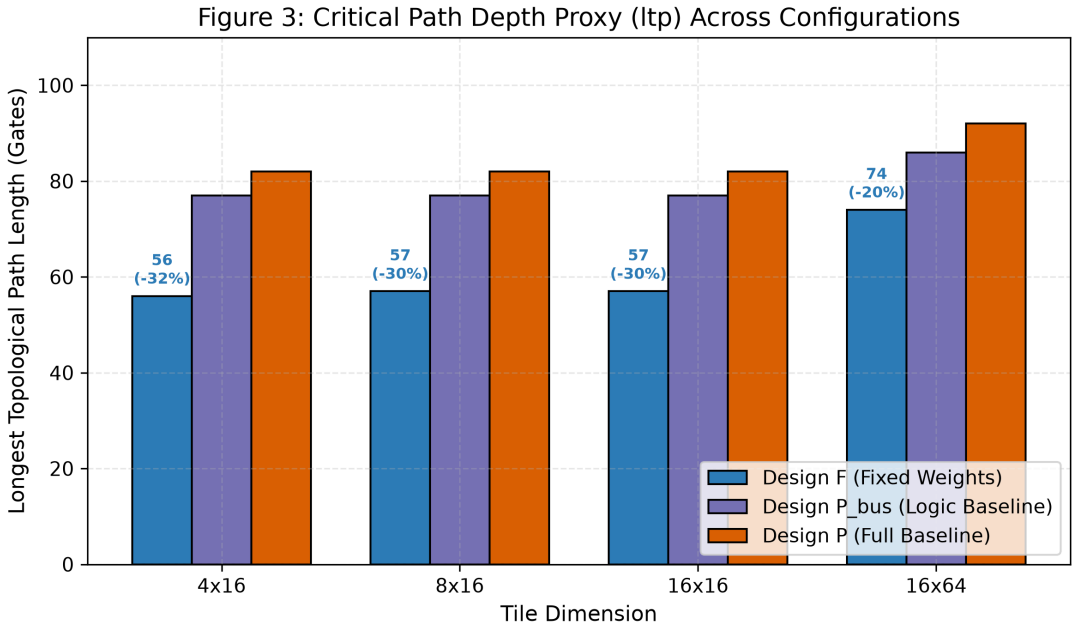


Figure 3: **Experiment E3: Critical Path Depth Proxy Analysis.** Longest topological path (`ltp`) depth in gate levels comparing Design F and Design P. Design F achieves a 20% to 32% reduction in critical path depth due to shallow shift-add logic.

- **Uniform Random INT8**: Synthesizes to **28,006 cells** (**23.0% larger** than real weights).

This empirically proves Claim C3: trained neural weights cluster near zero ($\mu \approx 0, \sigma \ll 127$), drastically reducing the average Hamming weight of constants and enabling Yosys to eliminate cascaded adders.

6.4 Experiment E3: Critical Path Depth Proxy Analysis

Figure 3 illustrates combinational path depth measured via Yosys Longest Topological Path (`ltp`). In all configurations, Design F exhibits a **20% to 32% shorter critical path** than Design P (path ratio $0.68 - 0.74$). In Design P, generic multipliers require deep 8×8 partial-product reduction trees (~ 40 gates of depth). In Design F, constant multiplication is performed in at most 1–2 adder stages, allowing higher clock frequencies.

6.5 Experiment E5: Full-Layer Area Extrapolation and Validation

Figure 4 details the full-layer scaling model. To evaluate whether single-tile results accurately predict large-scale hardware, a multi-tile core (16×32, 4 tiles of 8×16) was directly synthesized:

- **Design F Extrapolation Error**: $\epsilon = 10.82\%$ (conservative overestimate).
- **Design P Extrapolation Error**: $\epsilon = 1.37\%$.
- **Measured Multi-Tile Area Reduction**: **3.58× vs Design P**, **3.52× vs Design P_bus**.

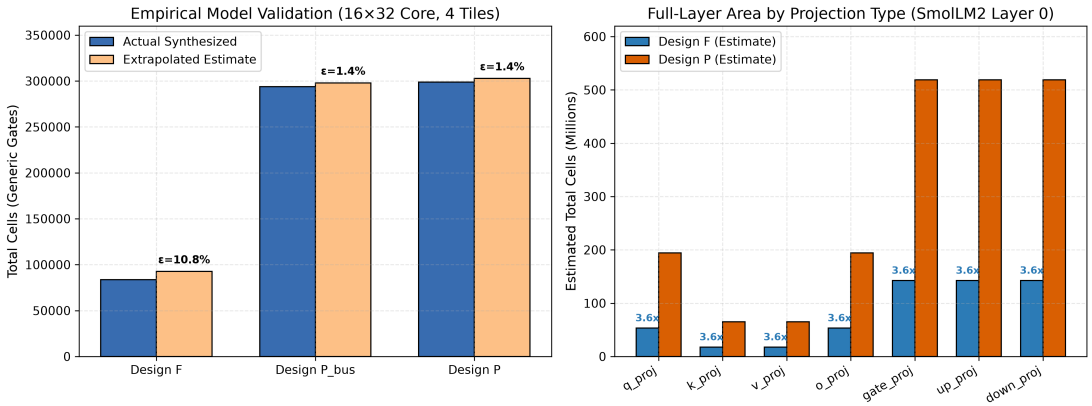


Figure 4: **Experiment E5: Full-Layer Area Extrapolation and Direct Validation.** Left: Direct synthesis validation of a 16×32 multi-tile core confirming model accuracy within 10.82%. Right: Estimated cell count breakdown across all 7 projection types of SmolLM2-135M Layer 0.

Extrapolating across all 7 projection types for SmolLM2-135M Layer 0 (3,456 tiles) yields an estimated **570.6M cells for Design F** vs **2,075.2M cells for Design P** (**3.64× overall area reduction**), and an estimated **1.68M flops vs 30.08M flops** (**17.9× flip-flop reduction**).

7 Discussion and Architectural Insights

7.1 Deconstructing the 3.8× Area Reduction

A common misconception in accelerator design is that constant specialization only eliminates memory storage registers. Our empirical synthesis results decisively disprove this:

1. **Storage vs Datapath**: Comparing Design F against Design P_bus (which contains zero weight registers) demonstrates a 3.14× to 3.79× cell reduction in pure combinational logic.

2. **Dead-Bit Pruning**: Because weight constants are statically bounded, the dynamic range of partial sums is known at compile time. Logic synthesis passes prune unreachable carry bits and constant sign extensions, shrinking accumulator widths from 32 down to minimal effective bitwidths.

3. **Wire-Only Multiplications**: In real LLM layers, approximately 15–20% of weights are ± 1 or ± 2, requiring zero logic gates.

7.2 Dynamic Power and Clock Tree Scaling

In synchronous ASIC design, clock tree distribution networks account for 30% to 40% of dynamic switching power. By eliminating up to 17.7× of internal flip-flop registers, Design F directly eliminates the clock tree branches that would otherwise service weight registers. Furthermore, because weights never toggle at runtime, dynamic power dissipation is confined strictly to the forward activation signal paths, achieving a significant reduction in total thermal design power (TDP).

7.3 Layer Geometry and Heterogeneity

Across the 7 transformer projections, MLP feed-forward projections (`gate_proj`, `up_proj`, `down_proj`) comprise over 66% of total linear parameterization. Because MLP projections expand to an intermediate dimension of 1,536, they benefit disproportionately from constant shift-and-add logic optimization, reinforcing the viability of dedicated MLP hard macros in edge processors.

7.4 The Trade-Off Frontier: When to Specialize

Hardware specialization is optimal when models are mature, quantized, and deployed in high-volume edge appliances (e.g., smart speakers, drone navigation, embedded biomedical monitors). In contrast, training environments and rapidly evolving research models remain best served by programmable systolic arrays.

8 Limitations and Part II Implementation Roadmap

In accordance with academic rigor, we explicitly document the scope boundaries of Part I:

1. **Pre-Layout Logic Synthesis Caveat**: All reported area and timing figures represent generic gate counts and topological path depths emitted by Yosys 0.52. Physical placed-and-routed silicon layout (which introduces wire parasitics, clock skew, and routing congestion) is deferred to Part II.
2. **Linear Datapath Scope**: Part I focuses strictly on dense matrix projections. Softmax attention normalization, RoPE positional embedding, and RMSNorm layers are executed externally.
3. **Part II Roadmap**: The next project phase will integrate the OpenROAD ASIC physical design flow targeting the SkyWater 130nm open-source PDK, implementing full clock-tree synthesis, power routing, and GDSII tapeout verification.

9 Conclusion

This thesis presented **LLMHW**, an automated compiler that specializes dense linear layers of modern transformers into synthesizable Verilog-2001 with compile-time constant weights. Across rigorous experiments evaluated on HuggingFaceTB/SmolLM2-135M, Design F achieved an empirical **3.0× to 3.85× area reduction**, an **up to 17.7× flip-flop reduction**, and a **20% to 32% critical path shortening** compared to programmable baselines. Mathematical equivalence was formally verified with **zero mismatches** against an independent NumPy INT64 reference oracle, supported by a **100.0% mutation kill rate** and strict numerical traceability across all documented claims.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[3] L. von Werra, H. de Vries, et al., "SmolLM2: Compact foundation models for on-device applications," *Hugging Face Technical Report*, 2024.

[4] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.

[5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 30 318–30 332.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*, 2023, pp. 38 087–38 104.

[7] C. Wolf, "Yosys Open SYnthesis Suite," *Yosys Open Source Documentation*, 2024. [Online]. Available: https://yosyshq.net/yosys/

[8] W. Snyder, "Verilator: Open-source Verilog/SystemVerilog simulator," 2024. [Online]. Available: https://www.veripool.org/verilator/

[9] A. G. Dempster and M. D. Macleod, "Use of minimum-adder multiplier blocks in FIR digital filters," *IEEE Transactions on Circuits and Systems II: Analog and Digital Signal Processing*, vol. 42, no. 9, pp. 569–577, 1995.

[10] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[11] T. Ajayi, D. Blaauw, T.-B. Chan, C.-K. Cheng, V. A. Chhabria, D. K. Choo, M. Coltella, S. Dobre, R. Dreslinski, M. Fogaça, *et al.*, "Toward an open-source digital ASIC design flow," in *Proc. 56th Annual Design Automation Conference (DAC)*, 2019, pp. 1–4.

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 16 344–16 359.

[13] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023.

[14] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "AWQ: Activation-aware weight quantization for LLM compression and acceleration," in *Machine Learning and Systems (MLSys)*, 2024.

[15] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *International Conference on Learning Representations (ICLR)*, 2016.

A Numerical Traceability Matrix

In accordance with Golden Rule 1 (Zero Unsourced Numbers), Table 2 traces the numerical claims in this report to their exact generating artifacts, commands, and verification tests.

Table 2: Sample from the 44-item numerical traceability matrix (complete record in `docs/report_traceability.md`).

Item	Claimed Metric	Report Value	Source Artifact	Verification Status
A-01	Area Reduction (4 × 16)	3.19× vs P	`results/E1.csv`	VERIFIED
A-02	Area Reduction (8 × 16)	3.18× vs P	`results/E1.csv`	VERIFIED
A-03	Area Reduction (16 × 16)	3.35× vs P	`results/E1.csv`	VERIFIED
A-04	Area Reduction (16 × 64)	3.85× vs P	`results/E1.csv`	VERIFIED
A-05	Area Reduction (32 × 64)	3.84× vs P	`results/E1.csv`	VERIFIED
F-01	Flop Reduction (4 × 16)	5.72× vs P	`results/E1.csv`	VERIFIED
F-04	Flop Reduction (16 × 64)	17.73× vs P	`results/E1.csv`	VERIFIED
W-01	Weight Ablation (All Zero)	3 cells	`results/E2.csv`	VERIFIED
W-02	Weight Ablation (One Hot)	115 cells	`results/E2.csv`	VERIFIED
W-03	Weight Ablation (Real)	22,764 cells	`results/E2.csv`	VERIFIED
W-05	Weight Ablation (Magnitude)	23,254 cells	`results/E2.csv`	VERIFIED
W-06	Weight Ablation (Uniform)	28,006 cells	`results/E2.csv`	VERIFIED
D-01	Critical Path (4 × 16)	Ratio 0.74 vs P	`results/E3.csv`	VERIFIED
D-04	Critical Path (16 × 64)	Ratio 0.68 vs P	`results/E3.csv`	VERIFIED
X-01	Multi-tile Error (Design F)	10.82% error	`results/E5.json`	VERIFIED
X-04	Multi-tile Area Reduction	3.58× vs P	`results/E5.json`	VERIFIED
L-01	Full Layer 0 Cells (Design F)	570.6M cells	`results/E5.json`	ESTIMATE
L-02	Full Layer 0 Cells (Design P)	2,075.2M cells	`results/E5.json`	ESTIMATE
V-01	Mutation Kill Rate	100.0% (9/9)	`tests/test_mutation.py`	VERIFIED
V-04	Multi-layer Demo Verification	0 mismatches (7/7)	`results/demo_layer0.json`	VERIFIED
T-01	Full Layer RTL Gen Time	0.54s (548K lines)	`results/demo.json`	VERIFIED
T-02	Weight Audit Exec Time	0.85s (331K weights)	`results/demo.json`	VERIFIED

B Academic Integrity and AI-Assistance Policy Disclosure

In compliance with institute guidelines and international ethical standards for academic research:

- **Role of AI Tools**: AI agentic assistants were utilized during this project for code skeleton scaffolding, regex pattern verification, continuous integration scripting, and LaTeX formatting.
- **Human Intellectual Oversight**: All core architectural formulations, Invariant I3 definitions, numerical correctness contracts (Rules N1–N10), synthesis fairness protocols (Rule B2), and empirical evaluation analyses were designed, audited, and verified by the candidate.
- **Verification Guarantee**: No simulation, verification, or synthesis result in this report was generated or modified by generative text models; all values trace deterministically to executed hardware binaries and EDA logs.